

NanoVDB `ReadAccessor` Benchmark — OLD vs NEW, CPU vs GPU

Benchmarks comparing the `NANOVDB_USE_OLD_ACCESSOR` caching behaviour (ON vs OFF) across access patterns, on CPU (single- and multi-threaded) and GPU. Results are collected from three machines: a **laptop**, a **desktop**, and a **workstation**.

What is being measured

The `ReadAccessor` caches the tree nodes visited on the previous access so a spatially nearby access can skip the traversal from the root.

- **OLD** (`NANOVDB_USE_OLD_ACCESSOR` defined): on a leaf-cache miss the level-1 and level-2 cache checks are compiled away (`if constexpr + else`), so the accessor falls straight to a full **root traversal**.
- **NEW** (`NANOVDB_NO_OLD_ACCESSOR` defined): all applicable cache levels are always checked, so a leaf-cache miss can still hit the **level-1 / level-2** cache instead of restarting at the root.

Methodology

What the numbers measure

Every reported figure is the **wall-clock time of a single random-access read** into the grid, i.e. one `accessor.getValue(ijk)` call, expressed in **nanoseconds per lookup** (ns/lookup). It is a pure *read-path* micro-benchmark: no values are written, no math is done on the result beyond a running sum (kept `volatile` / written to an output array so the compiler cannot elide the work).

- **CPU-1T** — one thread walks the coordinate list serially with a single accessor. This is a **latency** number: the cost of one dependent lookup.
- **CPU-MT** — the same total work split across all hardware threads (`nanovdb::util::forEach` → `tbb::parallel_for`), each grain using its own accessor. This is a **throughput** number: total time ÷ total lookups with all cores busy.
- **GPU** — the coordinate list is uploaded to the device; each thread processes a 32-coord chunk with its own accessor. Timed with CUDA events (kernel time only, excluding the one-time grid upload). Also a **throughput** number.

The only thing that differs between the OLD and NEW columns is the accessor's cache-fallback logic (a compile-time switch); the data, coordinates, and harness are identical.

The test data

A **fog-volume sphere** built with

```
nanovdb::tools::createFogVolumeSphere<float>(radius=500,  
voxelSize=1.0, halfWidth=3.0) . Its measured properties:
```

Property	Value
Value type	float
Index bounding box	<code>[-500, -500, -500] ... [500, 500, 500]</code> (1001 ³ voxels)
Dense voxel count of bbox	1,003,003,001 (~1.00 B)
Active voxels	523,592,077 (~523.6 M)
Fill ratio (active / dense bbox)	52.2 %
Upper internal nodes (level 2, 32 ³)	8
Lower internal nodes (level 1, 16 ³)	272
Leaf nodes (level 0, 8 ³)	82,608
Grid size in memory	188,484,640 bytes (~180 MB)

So it is a **fully populated solid ball** — the interior is all active voxels (the 52 % fill ratio is just the volume of a sphere inside its cube, $\pi/6 \approx 0.524$). It is *sparse* in the VDB sense (only the ball is stored, not the empty corners), but the region the benchmark actually samples is **100 % active**: all access patterns are confined to a cube well inside radius 500, so every lookup lands on a real leaf and traverses the tree for real — we are **not** measuring empty-space shortcuts, we are measuring genuine node traversal and cache reuse.

How much work, how it is averaged

Point patterns (Sequential / LeafJump / NodeJump / Random)	1,048,576 lookups each
Stencil	262,144 centres × 27 neighbours = 7,077,888 lookups
Sampled region	cube of half-extent 256 (points) / <code>[0, 64)</code> ³ dense cube (stencil), both fully inside the ball
Repetition	each configuration run 7 times; the median is reported (robust to OS jitter / turbo ramp)
Warm-up	GPU does one untimed warm-up launch before the 7 timed trials

Hardware / build

Two machines were benchmarked; results for each are in separate sections below.

	Laptop	Desktop	Workstation
CPU	32 HW threads; TBB parallel_for, 4096-coord grains	AMD Ryzen 9 9950X — 16 cores / 32 threads; same TBB settings	AMD Ryzen Threadripper PRO 7975WX — 32 cores / 64 threads; same TBB settings
GPU	NVIDIA RTX 5000 Ada Generation Laptop GPU (SM 8.9); 32-coord chunks/thread, 128 threads/block	NVIDIA RTX PRO 6000 Blackwell Max-Q Workstation Edition (SM 12.0); same CUDA settings	NVIDIA RTX 6000 Ada Generation (SM 8.9, 49 GB); same CUDA settings
Build	Release (-O3 -DNDEBUG), C+ +17 / CUDA 17	same	same
OLD vs NEW	selected at compile time per target (-DNANOVDB_USE_OLD_ACCESSOR vs - DNANOVDB_NO_OLD_ACCESSOR)	same	same

Access patterns

Pattern	Stride / shape	Cache behaviour
Sequential	1	leaf (level-0) cache always hot
LeafJump	8 (= leaf dim)	leaf cache cold; level-1 warm (NEW only)
NodeJump	128 (= lower-node dim)	leaf + level-1 cold; level-2 warm (NEW only)
Random	uniform in cube	all caches cold — both fall to root
Stencil	3×3×3 = 27 neighbours per centre, dense sweep	mostly same-leaf hits; boundary neighbours spill to level-1 (NEW rescues)

Results — Desktop (AMD Ryzen 9 9950X / RTX PRO 6000 Blackwell, SM 12.0)

Combined — ns per lookup (one `getValue` call), lower = faster

The stencil row is the per-lookup cost inside a 27-neighbour sweep.

Pattern	CPU-1T OLD	CPU-1T NEW	CPU- MT OLD	CPU- MT NEW	GPU OLD	GPU NEW
Sequential (stride 1)	2.45	1.44	0.26	0.18	0.035	0.020
LeafJump (stride 8)	2.06	1.41	0.21	0.16	0.035	0.020

Pattern	CPU-1T OLD	CPU-1T NEW	CPU- MT OLD	CPU- MT NEW	GPU OLD	GPU NEW
NodeJump (stride 128)	1.85	1.64	0.22	0.21	0.028	0.024
Random (uniform)	7.19	9.93	0.47	0.57	0.039	0.047
Stencil (3×3×3, 27- pt)	2.143	1.371	0.170	0.102	0.0587	0.0275

Legend: CPU-1T = single thread · CPU-MT = 32 threads (TBB) · GPU = RTX PRO 6000 Blackwell.

OLD → NEW speedup (>1 = the fix is faster)

Pattern	CPU-1T	CPU-MT	GPU
Sequential	1.70×	1.44×	1.75×
LeafJump	1.46×	1.31×	1.75×
NodeJump	1.13×	1.05×	1.17×
Random	0.72×	0.82×	0.83×
Stencil	1.56×	1.67×	2.14×

Detailed tables — Desktop

CPU, single-threaded (latency) — ns per lookup

Pattern	OLD	NEW	OLD → NEW
Sequential	2.45	1.44	1.70×
LeafJump	2.06	1.41	1.46×
NodeJump	1.85	1.64	1.13×
Random	7.19	9.93	0.72×
Stencil	2.143	1.371	1.56×

CPU, 32 threads (throughput) — ns per lookup

Pattern	OLD	NEW	OLD → NEW	MT speedup (NEW)
Sequential	0.26	0.18	1.44×	8.0×
LeafJump	0.21	0.16	1.31×	8.8×
NodeJump	0.22	0.21	1.05×	7.8×
Random	0.47	0.57	0.82×	17.4×
Stencil	0.170	0.102	1.67×	13.4×

GPU (throughput) — ns per lookup

Pattern	OLD	NEW	OLD → NEW
Sequential	0.035	0.020	1.75×

Pattern	OLD	NEW	OLD → NEW
LeafJump	0.035	0.020	1.75×
NodeJump	0.028	0.024	1.17×
Random	0.039	0.047	0.83×
Stencil	0.0587	0.0275	2.14×

Stencil (3×3×3) — reported per whole 27-neighbour stencil

Platform / Mode	OLD ns/stencil	NEW ns/stencil	OLD → NEW
CPU 1 thread	57.86	37.01	1.56×
CPU 32 threads	4.58	2.74	1.67×
GPU	1.5862	0.7413	2.14×

Fair platform comparison — NEW accessor, full hardware (ns per lookup)

Workload	CPU-1T	CPU-MT	GPU	GPU vs CPU-MT
Sequential	1.44	0.18	0.020	9.0×
Random	9.93	0.57	0.047	12.1×
Stencil	1.371	0.102	0.0275	3.7×

Results — Laptop (RTX 5000 Ada Generation Laptop GPU, SM 8.9)

Combined — ns per lookup (one `getValue` call), lower = faster

The stencil row is the per-lookup cost inside a 27-neighbour sweep.

Pattern	CPU-1T OLD	CPU-1T NEW	CPU-32T OLD	CPU-32T NEW	GPU OLD	GPU NEW
Sequential (stride 1)	3.57	1.96	0.56	0.46	0.058	0.058
LeafJump (stride 8)	3.82	2.21	0.55	0.49	0.057	0.057
NodeJump (stride 128)	3.94	3.05	0.58	0.53	0.054	0.055
Random (uniform)	11.47	15.32	0.82	1.04	0.088	0.095
Stencil (3×3×3, 27-pt)	4.61	2.13	0.322	0.191	0.086	0.040

Legend: CPU-1T = single thread · CPU-32T = 32 threads (TBB) · GPU = RTX 5000 Ada.

OLD → NEW speedup (>1 = the fix is faster)

Pattern	CPU-1T	CPU-32T	GPU
Sequential	1.82×	1.22×	1.00×
LeafJump	1.73×	1.12×	1.00×
NodeJump	1.29×	1.09×	0.98×
Random	0.76×	0.79×	0.93×
Stencil	2.17×	1.68×	2.17×

Detailed tables — Laptop

CPU, single-threaded (latency) — ns per lookup

Pattern	OLD	NEW	OLD → NEW
Sequential	3.57	1.96	1.82×
LeafJump	3.82	2.21	1.73×
NodeJump	3.94	3.05	1.29×
Random	11.47	15.32	0.76×
Stencil	4.61	2.13	2.17×

CPU, 32 threads (throughput) — ns per lookup

Pattern	OLD	NEW	OLD → NEW	MT speedup (NEW)
Sequential	0.56	0.46	1.22×	4.3×
LeafJump	0.55	0.49	1.12×	4.5×
NodeJump	0.58	0.53	1.09×	5.7×
Random	0.82	1.04	0.79×	14.7×
Stencil	0.322	0.191	1.68×	11.1×

GPU (throughput) — ns per lookup

Pattern	OLD	NEW	OLD → NEW
Sequential	0.058	0.058	1.00×
LeafJump	0.057	0.057	1.00×
NodeJump	0.054	0.055	0.98×
Random	0.088	0.095	0.93×
Stencil	0.086	0.040	2.17×

Stencil (3×3×3) — reported per whole 27-neighbour stencil

Platform / Mode	OLD ns/stencil	NEW ns/stencil	OLD → NEW
CPU 1 thread	124.48	57.49	2.17×
CPU 32 threads	8.70	5.17	1.68×
GPU	2.32	1.07	2.17×

Fair platform comparison — NEW accessor, full hardware (ns per lookup)

Workload	CPU-1T	CPU-32T	GPU	GPU vs CPU-32T
Sequential	1.96	0.46	0.058	7.9×
Random	15.32	1.04	0.095	10.9×
Stencil	2.13	0.191	0.040	4.8×

Results — Workstation (AMD Ryzen Threadripper PRO 7975WX / RTX 6000 Ada, SM 8.9)

Combined — ns per lookup (one `getValue` call), lower = faster

The stencil row is the per-lookup cost inside a 27-neighbour sweep.

Pattern	CPU-1T OLD	CPU-1T NEW	CPU-MT OLD	CPU-MT NEW	GPU OLD	GPU NEW
Sequential (stride 1)	3.20	1.64	0.20	0.21	0.034	0.024
LeafJump (stride 8)	3.09	1.65	0.20	0.22	0.035	0.023
NodeJump (stride 128)	3.18	2.90	0.20	0.25	0.027	0.025
Random (uniform)	8.35	11.42	0.28	0.40	0.046	0.052
Stencil (3×3×3, 27-pt)	3.492	1.981	0.143	0.078	0.0578	0.0269

Legend: CPU-1T = single thread · CPU-MT = 64 threads (TBB) · GPU = RTX 6000 Ada.

OLD → NEW speedup (>1 = the fix is faster)

Pattern	CPU-1T	CPU-MT	GPU
Sequential	1.95×	0.95×	1.42×
LeafJump	1.87×	0.91×	1.52×
NodeJump	1.10×	0.80×	1.08×
Random	0.73×	0.70×	0.88×
Stencil	1.76×	1.83×	2.15×

Detailed tables — Workstation

CPU, single-threaded (latency) — ns per lookup

Pattern	OLD	NEW	OLD → NEW
Sequential	3.20	1.64	1.95×

Pattern	OLD	NEW	OLD → NEW
LeafJump	3.09	1.65	1.87×
NodeJump	3.18	2.90	1.10×
Random	8.35	11.42	0.73×
Stencil	3.492	1.981	1.76×

CPU, 64 threads (throughput) — ns per lookup

Pattern	OLD	NEW	OLD → NEW	MT speedup (NEW)
Sequential	0.20	0.21	0.95×	7.8×
LeafJump	0.20	0.22	0.91×	7.5×
NodeJump	0.20	0.25	0.80×	11.6×
Random	0.28	0.40	0.70×	28.6×
Stencil	0.143	0.078	1.83×	25.4×

Note: for all point patterns the OLD accessor is faster in multi-threaded mode on this machine. See the cross-machine comparison below for analysis.

GPU (throughput) — ns per lookup

Pattern	OLD	NEW	OLD → NEW
Sequential	0.034	0.024	1.42×
LeafJump	0.035	0.023	1.52×
NodeJump	0.027	0.025	1.08×
Random	0.046	0.052	0.88×
Stencil	0.0578	0.0269	2.15×

Stencil (3×3×3) — reported per whole 27-neighbour stencil

Platform / Mode	OLD ns/stencil	NEW ns/stencil	OLD → NEW
CPU 1 thread	94.28	53.50	1.76×
CPU 64 threads	3.86	2.12	1.82×
GPU	1.5593	0.7266	2.15×

Fair platform comparison — NEW accessor, full hardware (ns per lookup)

Workload	CPU-1T	CPU-64T	GPU	GPU vs CPU-64T
Sequential	1.64	0.21	0.024	8.8×
Random	11.42	0.40	0.052	7.7×
Stencil	1.981	0.078	0.0269	2.9×

Cross-machine comparison (Laptop vs Desktop vs Workstation)

1. The stencil win is universal and consistent

The NEW accessor's stencil speedup is **~2× on GPU and 1.6–2.2× on CPU across all three machines**. This is the most reproducible finding — it does not depend on CPU generation, thread count, or GPU architecture.

2. GPU point-pattern speedup is GPU-model-dependent, not Ada vs Blackwell

Pattern	Laptop Ada RTX 5000 (SM 8.9)	Workstation Ada RTX 6000 (SM 8.9)	Desktop Blackwell RTX PRO 6000 (SM 12.0)
Sequential	1.00×	1.42×	1.75×
LeafJump	1.00×	1.52×	1.75×
NodeJump	0.98×	1.08×	1.17×
Random	0.93×	0.88×	0.83×

Both Ada machines show *different* results: the workstation RTX 6000 Ada extracts 1.42–1.52× for coherent point patterns while the laptop RTX 5000 Ada is neutral (1.00×). This suggests the speedup is not purely an SM generation effect — GPU model / power envelope / clock headroom also contribute. Blackwell (SM 12.0) remains the strongest at 1.75× for coherent patterns.

3. CPU MT point-pattern regression is unique to the high-thread-count Threadripper

Pattern	Laptop (32T) CPU-MT	Desktop (32T) CPU-MT	Workstation (64T) CPU-MT
Sequential	1.22×	1.44×	0.95×
LeafJump	1.12×	1.31×	0.91×
NodeJump	1.09×	1.05×	0.80×
Random	0.79×	0.82×	0.70×
Stencil	1.68×	1.67×	1.83×

On the Threadripper PRO (64 threads), the NEW accessor's extra cache-level checks become a net overhead in multi-threaded point access: all cache levels miss under random scatter across 64 competing threads, making the additional branch work pure cost. The stencil case, where each thread's access pattern is spatially coherent, still wins strongly (1.83×). Single-threaded CPU results remain consistent with the other machines (1.87–1.95× for coherent patterns).

4. Single-threaded CPU behaviour is consistent across all machines

Pattern	Laptop CPU-1T	Desktop CPU-1T	Workstation CPU-1T
Sequential	1.82×	1.70×	1.95×

Pattern	Laptop CPU-1T	Desktop CPU-1T	Workstation CPU-1T
LeafJump	1.73×	1.46×	1.87×
NodeJump	1.29×	1.13×	1.10×
Random	0.76×	0.72×	0.73×
Stencil	2.17×	1.56×	1.76×

All three machines show the same qualitative pattern: coherent access wins, random regresses. The absolute speedup varies slightly by CPU micro-architecture but the ranking is identical.

5. Random always regresses — hardware-independent in character

The regression is intrinsic: all caches miss anyway and the extra checks are pure overhead. On CPU-1T it is **0.72–0.76×** across all three machines. On GPU it is **0.83–0.93×**. This confirms the regression is a property of the access pattern, not the hardware.

6. The qualitative pattern is fully reproducible across machines

Same ranking, same sign on single-threaded CPU and GPU rows across all three machines. The three independent data sets confirm each other's conclusions.

Bottom line: the fix is solidly beneficial and portable. Stencil is the universal ~2× win on all platforms. GPU coherent-pattern gains depend on GPU model (neutral on RTX 5000 Ada laptop, 1.42–1.52× on RTX 6000 Ada workstation, 1.75× on Blackwell). The one nuance is that on very high thread-count CPUs (64T Threadripper PRO), multi-threaded point-pattern throughput regresses slightly — the extra cache checks cost more than they save when 64 threads scatter across the grid — while the stencil win remains large (1.83×).

Per-machine takeaways

- **Stencil access is the biggest and most universal win**, consistently ~2× on GPU and 1.6–2.2× on CPU across all three machines. Each centre does 27 correlated lookups; boundary-crossing neighbours that the OLD accessor sends to the root are caught by the NEW accessor's level-1 cache, and the saving compounds across the 27.
- **Coherent point access benefits on single-threaded CPU** (1.1–1.95× depending on machine). GPU gains for coherent point patterns are model-dependent: neutral on the RTX 5000 Ada laptop (1.00×), moderate on the RTX 6000 Ada workstation (1.42–1.52×), and highest on Blackwell (1.75×). Both Ada GPUs share the same SM 8.9 architecture, so the difference is attributable to power/thermal headroom and GPU model rather than SM generation alone.
- **High thread-count CPU (Threadripper PRO, 64T) shows MT regression for point patterns.** With 64 threads scattering across the grid, the NEW accessor's extra cache-level checks become pure overhead: all levels miss anyway and the additional branches reduce

throughput by 5–30%. The stencil case is unaffected — each thread remains spatially coherent and the 1.83× win holds.

- **Only pathological fully-random access regresses** (~0.7–0.8× on single-threaded CPU, ~0.83–0.93× on GPU), because every cache level misses anyway and the extra checks are pure overhead. This is hardware-independent.
- **The multi-threaded CPU makes the GPU comparison fair.** Against a fully loaded 32-core CPU the GPU is ~4–12× faster (not the misleading 30–170× a single thread would suggest). The gap is smallest for the stencil, where heavy cache reuse plays to the CPU's larger per-core caches.
- **CPU multi-thread scaling varies by platform:** coherent patterns scale ~4–9× on 32 cores (laptop/desktop); on the 64-thread Threadripper PRO the stencil achieves ~25× scaling while point patterns are memory-bandwidth-limited at ~8×. Random access scales best (~28–30×) across all platforms because its high per-lookup latency gives more threads useful overlap.

Building & running

```
# Configure with the benchmark (and CUDA, for the GPU variant) enabled
cmake -S . -B build -DNANOVDB_BUILD_BENCHMARK=ON -DNANOVDB_USE_CUDA=ON

# Build all four executables
cmake --build build --target \
    bench_accessor_old bench_accessor_new \
    bench_accessor_cuda_old bench_accessor_cuda_new -j

# Run (CPU prints single- and multi-threaded columns; GPU prints device throughput)
cd build/nanovdb/nanovdb/benchmark
./bench_accessor_old      # CPU, OLD accessor
./bench_accessor_new      # CPU, NEW accessor
./bench_accessor_cuda_old # GPU, OLD accessor
./bench_accessor_cuda_new # GPU, NEW accessor
```

The OLD vs NEW behaviour is selected at compile time per target (-DNANOVDB_USE_OLD_ACCESSOR vs -DNANOVDB_NO_OLD_ACCESSOR), so no reconfigure is needed to compare them.

Files

File	Purpose
BenchPatterns.h	Shared access-pattern generation (identical coords for CPU & GPU)
BenchAccessor.cc	CPU benchmark (single- and multi-threaded)
BenchAccessorCuda.cu	GPU (CUDA) benchmark
CMakeLists.txt	Builds the OLD/NEW × CPU/GPU targets